# Autoregressive Temporal Knowledge Graph Generation

**Jaden Cohen, Tony Chang, Nicholas Nevin Tan, Victoria Duran-Valero, Will Du**

https://github.com/Invictus108/Temporal_Knowledge_Graph_GNN

https://www.youtube.com/watch?v=J3knMh1oDCc

## Abstract

Temporal knowledge graphs (TKGs) encode how relational facts between entities evolve over time. This makes them a natural representation for domains such as geopolitical events and curated encyclopedic knowledge. Most existing work frames TKG learning as a completion or link-prediction task, ranking which facts will hold at a future timestep, but does not explicitly model the graph as a sequence of edits. We propose an autoregressive model that directly predicts graph deltas: the set of edges added to and deleted from a knowledge graph between consecutive timestamps. Our architecture combines text-informed node representations, a multi-head attention graph neural network (GNN) for structural encoding, and per-node LSTM states for temporal memory. A budgeted candidate-proposal system constrains the combinatorial search space at each step, after which separate addition and deletion scoring heads produce calibrated edge probabilities. Each head combines a DistMult-style relational scoring with an MLP over explicit temporal features. We train end-to-end with binary cross entropy and truncated backpropagation through time (BPTT). Evaluation on controlled synthetic TKG benchmarks shows strong ranking performance (Hits@10 up to 0.82 for additions and 1.00 for deletions on simple dynamics), with partial transfer to the real-world Wikidata12k dataset. Our results demonstrate that treating graph evolution as a delta-prediction problem is a tractable and expressive formulation for temporal knowledge graph forecasting.

## 1 Introduction / Motivation

Knowledge graphs represent facts about the world as triples (subject, relation, object) and have become a foundational data structure for question answering and information retrieval. Real-world knowledge, however, is not static. Political relationships form and dissolve. Scientific collaborations begin and end. Facts that were true last year may no longer be true today. Temporal knowledge graphs (TKGs) capture this dynamism by attaching timestamps to facts, producing a sequence of graph snapshots that record how the world changes over time.

The central challenge in TKGs is predicting future graph states from past ones. The dominant framing in prior work is completion or link prediction: given a query triple with a missing entity (e.g., (United States, hasLeader, ?)), rank all candidate entities by the probability that the triple is true at a target timestep. This framing is useful but limited in a key way because it treats each candidate triple in isolation and does not model the structural changes between consecutive snapshots. Models trained this way usually score static plausibility instead of predict graph dynamics.

We take a different approach. We argue that TKG forecasting is fundamentally a graph editing problem: at each timestep, a graph transitions to its successor by a set of edge additions and edge

Preprint. Under review.

deletions. Predicting these deltas directly is a more faithful description of how real-world knowledge evolves, and it allows for autoregressive rollout by applying predicted edits to produce a next snapshot, then predicting edits again (which static completion models cannot do).

This framing introduces many non-trivial challenges. Firstly, the space of candidate edges is large. With tens of thousands of entities and dozens of relation types, enumeration is intractable. Additionally, addition and deletion have different statistical structures; additions depend on rich relational and historical context (what edges are about to form?), while deletions depend mostly on edge persistence (what edges have been present long enough to expire?). Finally, real TKGs are noisy and incomplete, making clean temporal signals hard to exploit.

Our model addresses each of these challenges. A budgeted proposal system chooses a tractable candidate set using diverse structural, similarity-based, and historical criteria. Separate scoring heads for additions and deletions capture their different statistical profiles. And a combination of GNN-based structural encoding, LSTM-based temporal memory, and explicit hand-crafted history features gives the model multiple additional signals for each candidate edge.

## 2 Background / Related Work

Temporal knowledge graphs (TKGs) extend static knowledge graphs by associating relational facts with time, making them a natural representation for evolving domains such as geopolitical events, scientific collaboration, and curated knowledge bases like Wikidata. Learning useful representations from TKGs, therefore, requires models that can exploit the relational structure within each time step while also capturing how entities and relations change across time. In the following, we review three representative approaches that illustrate complementary strategies for this problem: autoregressive temporal reasoning over event sequences (RE-NET), explicit temporal message passing for knowledge graph completion (TeMP), and memory-based modeling of continuous-time dynamic interaction graphs (TGN). Together, these works motivate our design choices for combining graph-based encoders with mechanisms that summarize historical context when predicting graph evolution.

**RE-NET (Jin et al., EMNLP 2020)** Recurrent Event Network (RE-NET) treats temporal knowledge graph reasoning as an autoregressive prediction of future facts from a history of past graphs. It combines a recurrent event encoder that summarizes temporal sequences of interactions with a neighborhood aggregation module that captures relational context among co-occurring facts at a given time, enabling multi-step inference over future timestamps. This framing is especially relevant when the learning objective is to model how the graph evolves rather than only scoring static triples.[Jin et al., 2020]

**TeMP (Wu et al., EMNLP 2020)** TeMP addresses temporal knowledge graph completion by explicitly integrating multi-relational message passing over recent graph snapshots with a temporal dynamics model (e.g., recurrent or self-attention-based) that aggregates information across time. The method is designed to leverage both multi-hop structure and recent temporal context, and it additionally targets practical TKG challenges such as temporal sparsity and variable entity activity across timesteps.[Wu et al., 2020]

**TGN (Rossi et al., 2020)** Temporal Graph Networks (TGN) provide a general-purpose framework for learning on continuous-time dynamic graphs represented as streams of time-stamped events. TGN combines node memory (a compressed history state) with graph-based operators and time-aware embeddings, and it is typically evaluated on interaction prediction tasks where efficient online updates matter. Although not TKG-specific, the TGN memory-and-message pass design is a useful reference for modeling long-range temporal dependencies alongside local graph neighborhoods.[Emanuele Rossi, 2020]

In contrast to generic memory models like TGN, we explicitly model graph evolution as two coupled delta processes (i.e., edge deletions and edge additions), and make forecasting tractable by combining (i) a structural message-passing encoder on the current snapshot, (ii) a temporal memory (in the RE-NET–inspired variant, updated event-driven rather than globally), and (iii) a budgeted candidate-proposal system that constrains the combinatorial search space for additions while still leveraging multi-hop structure, similarity, and historical recurrence. This "predict edits, not just facts" framing directly targets multi-step rollout of future graphs and makes deletions a first-class prediction target, which the related models typically only handle implicitly.

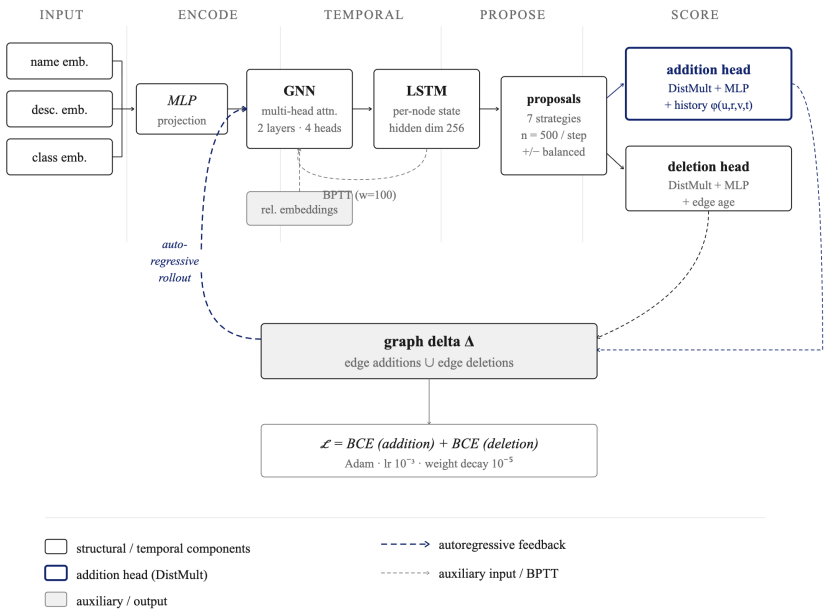


Figure 1: Model schematic. At each timestep, text-informed node embeddings are encoded via a multi-head graph neural network and per-node LSTM states, then passed to a budgeted candidate proposal system; separate addition and deletion scoring heads produce edge probabilities, and predicted graph deltas are fed back autoregressively into the next timestep.

# 3 Methods / Model

## 3.1 Architecture

We propose an autoregressive temporal knowledge graph (TKG) model that predicts *changes* (edge additions and deletions) over time. The model combines text-informed node representations, an attention-based graph neural network (GNN), and node-wise recurrent states for temporal reasoning.

## 3.2 Node and Edge Representations

Each node is initialized from multiple text-derived embeddings (e.g., name, description, class), which are concatenated and projected into a shared latent space via an MLP:

$$h_i^{(0)} = \text{MLP}([\mathbf{e}_{\text{name}}, \mathbf{e}_{\text{desc}}, \mathbf{e}_{\text{class}}]).$$

This incorporates semantic information beyond graph structure, enabling the model to generalize across sparsely connected nodes and capture similarity between entities with limited interaction history.

Edges are drawn from a fixed relation vocabulary with learned embeddings.

## 3.3 Graph and Temporal Encoding

Node representations are updated via attention-based message passing:

$$h_i^{(t+1)} = \sigma\left(\sum_{j \in \mathcal{N}(i)} a_{ij} \cdot m_{ij}\right), \quad m_{ij} = W_h h_j^{(t)} + W_r r_{ij},$$

where attention weights are computed over neighbors:

$$a_{ij} = \text{softmax}_j\left(\sigma(a(z_i, z_j, r_{ij}))\right), \quad z_i = W_n h_i^{(t)}.$$

Attention allows the model to weight neighbors dynamically, which is particularly important in temporal graphs where the relevance of interactions varies over time.

We use residual connections and layer normalization to stabilize training. Multi-head attention can be applied by computing independent heads and concatenating outputs.

To model temporal dynamics, each node maintains a recurrent hidden state:

$$\mathbf{s}_i^{(t)} = \text{LSTM}(h_i^{(t)}, \mathbf{s}_i^{(t-1)}),$$

which captures historical interaction patterns and is used during scoring. This enables the model to maintain persistent temporal context beyond a single snapshot, capturing long-term dependencies and recurring interaction patterns.

### 3.4 Change Prediction via Candidate Scoring

At each timestep, the model predicts graph updates by scoring candidate edges for either **addition** or **deletion**. Enumerating all possible edges is computationally intractable, so we construct a candidate set consisting of:

- **Positive samples**: true additions and deletions observed at the next timestep
- **Negative samples**: an equal number of sampled non-events

to mitigate class imbalance, which would otherwise bias the model toward predicting non-events.

**Negative sampling.** For addition candidates, negatives are generated via multiple strategies:

- **Two-hop proposals:** propose edges between nearby nodes, following classical link prediction intuition [Liben-Nowell and Kleinberg, 2007].
- **Similarity proposals:** propose edges between nodes with similar graph and temporal states.
- **Historical proposals:** reuse edges that appeared in recent snapshots but are currently absent.
- **Recurrence proposals:** prioritize past edges that were frequent or recent [Zhu et al., 2021, Li et al., 2022].
- **Activity-biased proposals:** sample more often around historically active nodes.
- **Relation-conditioned proposals:** select a source and relation, then retrieve likely destinations [Jin et al., 2020, Wang et al., 2023].
- **Random proposals:** include a small random set for diversity.

These strategies balance plausibility and diversity, ensuring that negatives are both challenging and representative of the underlying temporal dynamics.

For deletion candidates, negatives correspond to edges that persist (i.e., not deleted).

### 3.5 Scoring Heads

We use separate scoring heads for addition and deletion, reflecting their different statistical structure.

**Addition head.** The addition head predicts whether a new edge $(u, r, v)$ should be formed. It receives:

- Node embeddings $h_u, h_v$
- Relation embedding $r$
- Node temporal states $\mathbf{s}_u, \mathbf{s}_v$
- A structured feature vector $\phi(u, r, v, t)$ capturing historical statistics

The feature vector includes:

- Pair-level features: past interactions, recency, frequency, and streaks
- Triple-level features: $(u, r, v)$ occurrence statistics
- Node-level features: activity of $u$ and $v$
- Structural features: common neighbors and higher-order connectivity

All count-based features are log-scaled. These features capture multi-scale temporal statistics (pair, triple, and node level), which are predictive in temporal link prediction tasks.

**Deletion head.** The deletion head predicts whether an existing edge should be removed. In contrast to the addition head, it uses a minimal feature set:

- Node embeddings $h_u, h_v$
- Relation embedding $r$
- Node temporal states $\mathbf{s}_u, \mathbf{s}_v$
- Edge age (time since last observed)

This reflects the hypothesis that deletion dynamics depend primarily on persistence, unlike additions which rely on richer relational and historical context.

**Scoring function.** Both heads use an MLP to produce a probability:

$$\hat{y} = \sigma\left(\text{MLP}(h_u, h_v, r, \mathbf{s}_u, \mathbf{s}_v, \cdot)\right),$$

where the inputs differ as described above.

### 3.6 DistMult Scoring

To encourage the model to capture relational structure in its scoring, we used a DistMult-inspired interaction term as a component of the triple feature vector fed to both scoring heads. DistMult (Yang et al., 2015) is a bilinear knowledge graph embedding model that scores a triple (u, r, v) as the element-wise product of the subject, relation, and object embeddings, summed to a scalar:

$$f(u, r, v) = \langle \mathbf{e}_u, \mathbf{e}_r, \mathbf{e}_v \rangle = \sum_k e_{u,k} \cdot e_{r,k} \cdot e_{v,k}. \tag{1}$$

DistMult captures meaningful relational structure because a high score requires alignment between the subject, relation, and object embeddings across the same latent dimensions, encoding a form of type-level compatibility between endpoint entities and the relation type.

Rather than using DistMult in our model as a standalone scoring function, we integrate its interaction signal directly into the feature vector passed to the MLP-based scorer. After projecting the source and destination node states into a shared relational space via a learned linear projection $\mathbf{W}_n$, and embedding the relation via $\mathbf{e}_r$, we construct the feature vector:

$$\mathbf{f}(u, r, v) = \left[\, \mathbf{h}_u^{\text{proj}},\ \mathbf{e}_r,\ \mathbf{h}_v^{\text{proj}},\ \mathbf{h}_u^{\text{proj}} \odot \mathbf{h}_v^{\text{proj}},\ \left|\mathbf{h}_u^{\text{proj}} - \mathbf{h}_v^{\text{proj}}\right| \,\right], \tag{2}$$

where $\odot$ refers to element-wise multiplication. The term $\mathbf{h}_u^{\text{proj}} \odot \mathbf{h}_v^{\text{proj}}$ is the DistMult interaction term with the relation embedding implicitly folded into the projection space. Putting it alongside the difference vector $|\mathbf{h}_u^{\text{proj}} - \mathbf{h}_v^{\text{proj}}|$ gives the MLP a multiplicative compatibility signal and an asymmetric distance signal. This allows the scorer to learn complementary interaction patterns without committing to a single inductive bias.

For the addition head, this feature vector is further extended with an explicit history projection $\boldsymbol{\phi}(u, r, v, t) \in \mathbb{R}^{18}$ encoding pair-level, triple-level, node-level, and structural statistics (see Section **??**), giving the addition scorer access to both implicit relational compatibility via DistMult and explicit temporal evidence. The deletion head uses the base feature vector without history features, reflecting the hypothesis that deletion depends primarily on edge persistence rather than relational plausibility.

### 3.7 Loss Function

We optimize binary cross entropy over both addition and deletion candidates:

$$\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N}\left[y_i \log(\hat{y}_i) + (1 - y_i)\log(1 - \hat{y}_i)\right].$$

Losses from addition and deletion heads are summed.

### 3.8 Autoregressive Generation

At inference time, the model generates graph updates autoregressively:

- Construct candidate edges
- Score addition and deletion probabilities
- Sample or threshold updates
- Apply changes and feed the updated graph into the next timestep

This autoregressive formulation allows predicted updates to influence future graph evolution, capturing feedback effects over time.

## 4 Empirical Results

### 4.1 Experimental Setup

We evaluate the proposed model on controlled synthetic temporal knowledge graph (TKG) datasets and on Wikidata12k. The synthetic datasets are designed to isolate key challenges in temporal graph reasoning, including simple temporal dynamics, multi-stage interaction patterns, and structural constraints. These datasets provide controlled environments where ground-truth temporal processes are known, enabling targeted evaluation of model behavior.

We focus on ranking-based evaluation using Hits@k (k = 1, 3, 10), which measures the model's ability to rank true edge updates among candidate sets. This is the standard evaluation protocol for temporal knowledge graph completion. We additionally report training and validation loss for the addition and deletion heads to assess convergence.

Table 1: Summary of synthetic datasets.

| Dataset | # Entity Types | # Relations | Primary Challenge |
|---|---|---|---|
| Simple-TKG | 2 | 2 | Basic temporal dynamics |
| MultiPath-TKG | 2 | 5 | Multi-stage temporal reasoning |
| Structured-TKG | 5 | 5 | Structural constraints |

### 4.2 Synthetic Dataset Results

We evaluate performance separately for edge additions and deletions, as these tasks exhibit different statistical structure.

#### 4.2.1 Simple-TKG

Simple-TKG serves as a sanity check. It consists of a single deterministic interaction process between two entity types, serving as a baseline for learning basic temporal edge dynamics. The model converges rapidly, with decreasing loss and increasing Hits@k for both addition and deletion, indicating successful learning of simple temporal dynamics.

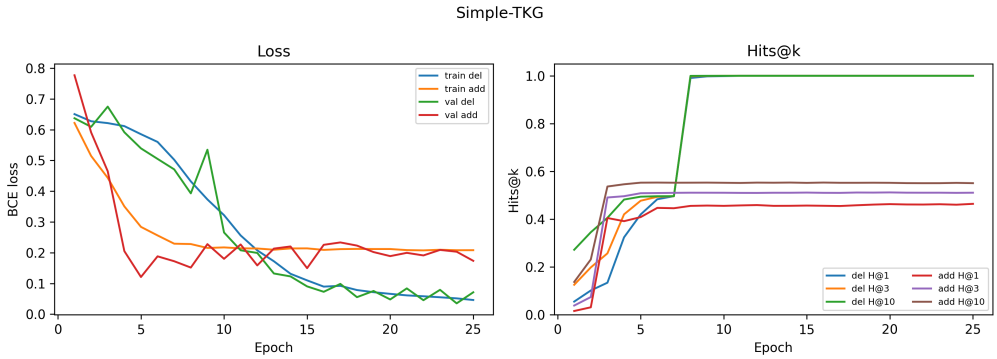


Figure 2: Performance on Simple-TKG. The model converges rapidly and achieves near-perfect ranking performance, reflecting the simplicity of the underlying deterministic temporal process.

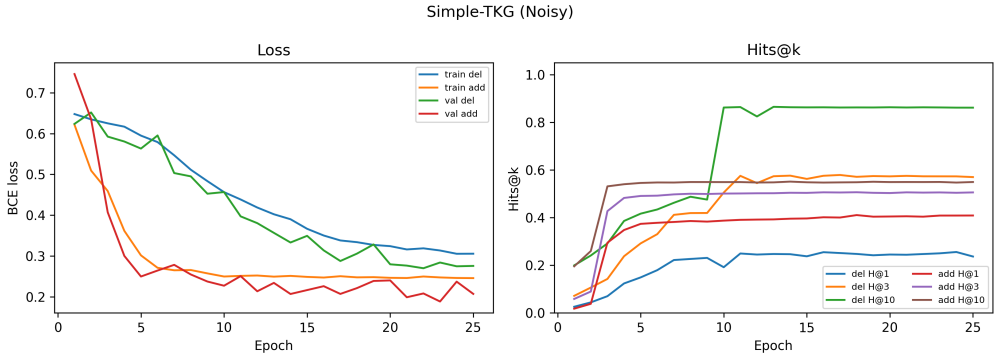


Figure 3: Performance on noisy Simple-TKG. Noise significantly degrades deletion performance while addition remains relatively stable, highlighting the sensitivity of deletion prediction to stochastic dynamics.

### 4.2.2 MultiPath-TKG

MultiPath-TKG introduces multiple competing temporal trajectories between entity pairs, requiring the model to distinguish between alternative multi-stage interaction patterns. The model maintains strong ranking performance, indicating effective learning of multi-stage temporal dependencies.

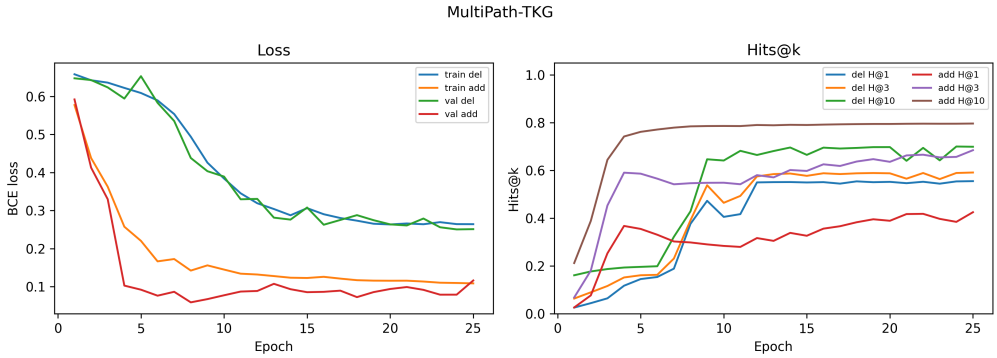


Figure 4: Performance on MultiPath-TKG. The model achieves strong ranking performance despite multiple competing temporal trajectories, indicating effective learning of multi-stage interaction patterns.

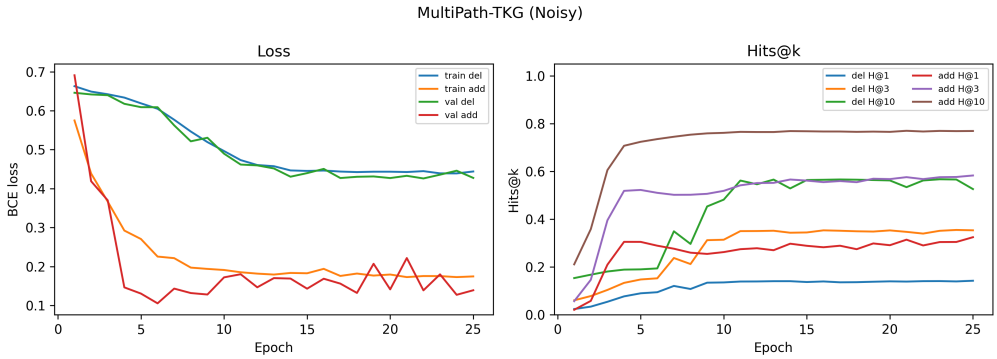


Figure 5: Performance on noisy MultiPath-TKG. Although noise introduces ambiguity between competing temporal paths, the model continues to improve in Hits@k, demonstrating robustness to complex temporal structure.

### 4.2.3 Structured-TKG

Structured-TKG imposes type-level constraints on valid edges, requiring the model to learn temporal dynamics while respecting global graph structure. The model performs well, demonstrating sensitivity to both temporal and structural information.

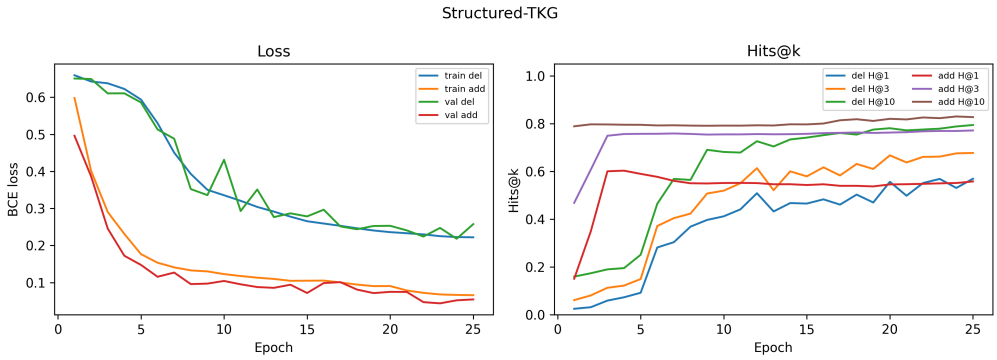


Figure 6: Performance on Structured-TKG. The model shows consistent loss reduction and strong improvements in Hits@k, indicating effective learning of temporal dynamics under structural constraints.

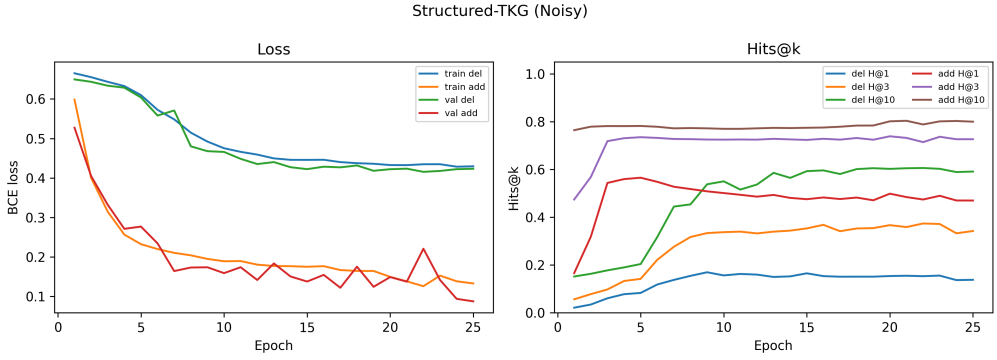


Figure 7: Performance on noisy Structured-TKG. While noise slows convergence and reduces ranking performance, the model continues to improve over training, demonstrating robustness to structural and stochastic complexity.

### 4.3 Summary of Synthetic Performance

We report final ranking performance in Table 2.

Table 2: Final synthetic dataset performance (Hits@k).

| Dataset | Noise | Add H@1 | Add H@3 | Add H@10 | Del H@1 | Del H@3 | Del H@10 |
|---|---|---|---|---|---|---|---|
| Simple-TKG | No | 0.46 | 0.51 | 0.55 | 1.00 | 1.00 | 1.00 |
| Simple-TKG | Yes | 0.40 | 0.60 | 0.54 | 0.23 | 0.57 | 0.86 |
| MultiPath-TKG | No | 0.43 | 0.68 | 0.79 | 0.55 | 0.59 | 0.69 |
| MultiPath-TKG | Yes | 0.31 | 0.55 | 0.76 | 0.13 | 0.33 | 0.53 |
| Structured-TKG | No | 0.53 | 0.74 | 0.82 | 0.54 | 0.63 | 0.77 |
| Structured-TKG | Yes | 0.50 | 0.72 | 0.80 | 0.12 | 0.34 | 0.60 |

Across synthetic datasets, the model shows strong ranking performance and robustness to noise. Addition performance remains high even under noise, while deletion performance degrades more significantly.

These results validate that the model effectively captures temporal dynamics across settings, including multi-path interaction patterns and structurally constrained graphs.

### 4.4 Real-World Evaluation on Wikidata12k

We evaluate the model on Wikidata12k, which contains noisy and incomplete interactions. While absolute performance is lower, Hits@k improves steadily over training.

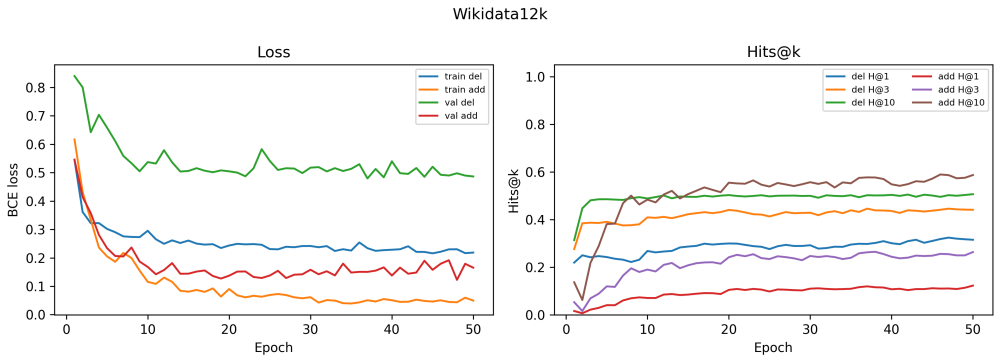


Figure 8: Performance on Wikidata12k. The model exhibits steady improvements in ranking performance despite higher noise and slower convergence compared to synthetic datasets.

Table 3: Final performance on Wikidata12k (Hits@k).

| Dataset | Add H@1 | Add H@3 | Add H@10 | Del H@1 | Del H@3 | Del H@10 |
|---|---|---|---|---|---|---|
| Wikidata12k | 0.12 | 0.26 | 0.58 | 0.32 | 0.44 | 0.50 |

Performance on Wikidata12k is lower than on synthetic datasets, which is expected given its noisier, sparser, and incomplete nature. Unlike the controlled setting, real-world data contains missing and irregular observations, where the absence of an edge does not necessarily indicate a true deletion. This makes deletion prediction particularly challenging: for example, if a member of parliament dies or leaves office, the corresponding edge simply disappears, providing no explicit signal to the model. Moreover, many entities share similar relational contexts, making it difficult to distinguish which specific edge should be removed. As a result, deletion requires inferring unobserved events from weak or ambiguous signals, whereas addition relies on more direct positive evidence. Despite

these challenges, the model still achieves steady improvements in Hits@k, indicating that it captures meaningful temporal structure even in realistic settings.

### 4.5 Discussion

The model effectively learns temporal graph dynamics in controlled settings and exhibits partial generalization to real-world data.

Across synthetic datasets, addition prediction remains strong even under noise, while deletion prediction degrades more substantially. This asymmetry is expected: addition relies on detecting positive temporal signals, whereas deletion depends on the absence of future interactions, making it inherently more sensitive to noise and incomplete observations. This suggests that modeling deletions may require richer contextual signals, such as relation-specific lifetimes, latent state changes, or external temporal cues.

The performance gap between synthetic and real-world datasets highlights the challenges of temporal knowledge graph forecasting in practice, where data is noisy, sparse, and non-stationary. These results suggest that while the proposed architecture captures structured temporal patterns, improving robustness to missing and ambiguous signals remains an important direction for future work.

The reliance on fixed heuristics may limit performance if true edges are not proposed or if negatives are not optimally challenging. Learning the proposal distribution or incorporating adaptive hard-negative mining could further improve both recall and ranking accuracy.

Additionally, the asymmetry between addition and deletion performance is reflected in the behavior of the DistMult-style interaction term across the two scoring heads. For additions, relational compatibility between candidate endpoints is a meaningful prior, in other words an edge is more likely to form if the source, relation, and destination align in the projected embedding space. The element-wise product captures exactly this signal. For deletions, however, an edge that already exists is by definition relationally plausible, meaning the head relies more on edge age and node temporal states than on relational compatibility. This overall suggests that DistMult contributes more to addition scoring than deletion.

This asymmetry was also visible in training becausethe addition head converged faster and to a lower loss, while the deletion head exhibited slower and noisier dynamics, especially under noise and on Wikidata12k. Truncated BPTT with a window of 100 timesteps was essential to maintain stable gradient flow across the long temporal sequences, which showed us a practical trade-off between getting long-range dependencies and avoiding vanishing gradients. And lastly, our experience building a preprocessing pipeline for ICEWS18 anticipated many of the real-world challenges observed on Wikidata12k: larger and noisier vocabularies, misaligned temporal granularity, implicit deletion signals where an interaction stops being reported instead of marked as removed. These challenges informed our decision to standardize on Wikidata12k and highlight a broader limitation of the current approach.

Overall, these results support the effectiveness of the autoregressive delta-prediction framework.

## 5 Conclusions / Future Work

We presented an autoregressive TKG model that predicts graph edits (additions and deletions) between consecutive snapshots rather than scoring static triples. The model combines text-derived node embeddings, an attention-based GNN, per-node LSTM states, and a budgeted candidate-proposal system, with separate heads for additions and deletions. On synthetic benchmarks the model ranks true edits well (Hits@10 up to 0.82 for additions and 1.00 for deletions in the noiseless case), and on Wikidata12k it improves steadily over training despite the noisier setting. Overall, framing TKG forecasting as edit prediction seems like a workable alternative to standard completion.

A few directions stand out for future work. The largest gap is on Wikidata12k, where deletions are especially hard because a missing observation is not necessarily a real deletion; loss functions that handle missing-not-at-random data, or pretraining the encoder on static Wikidata before temporal fine-tuning, would likely help. The candidate proposal system is currently a fixed mix of heuristics, and learning the proposal distribution, or doing hard-negative mining based on current model errors,

is a natural next step. Adding an explicit type system would shrink the candidate space and give the addition head a stronger prior, particularly on real data. Finally, our nodes use only a small set of text fields; richer context from larger pretrained encoders, or from structured attributes that change over time, should improve performance on sparsely connected and newly appearing entities.

## Graduate Student Contributions

**Nicholas Nevin Tan:** was primarily responsible for the design, implementation, and analysis of the addition and deletion proposal strategies. His work focused on constructing positive and negative candidate pools using structural, temporal, and relation-aware heuristics, improving proposal quality and recall through iterative experimentation, and conducting background research on prior temporal knowledge graph and link prediction literature to motivate and refine these strategy choices.

**Victoria Duran-Valero:** was primarily responsible for the integration of DistMult-style relational scoring into the triple-scoring heads, the design and implementation of the loss function and optimization procedure, and the development of a data pipeline to evaluate compatibility with the ICEWS18 benchmark. Her work focused on incorporating DistMult's element-wise interaction signal as an explicit feature within the MLP-based scorer alongside difference-based features, implementing binary cross-entropy training with truncated BPTT and Adam optimization, and adapting the model's preprocessing pipeline to the ICEWS18 dataset format as a candidate alternative to Wikidata12k.

**Tony Chang:** was primarily responsible for reviewing research papers that performed similar or related work to both incorporate into this project's approach and compare. His work focused on analyzing each paper's architecture and results to find key differences between their design and this project's design in order to demonstrate the novelty of the project's architecture. Also assisted with data tag gathering and formatting of data and the paper.

## References


Fabrizio Frasca Davide Eynard Federico Monti Michael M. Bronstein Emanuele Rossi, Ben Chamberlain. Temporal graph networks for deep learning on dynamic graphs. *CoRR*, abs/2006.10637, 2020. URL `https://arxiv.org/abs/2006.10637`.

Woojeong Jin, Meng Qu, Xisen Jin, and Xiang Ren. Recurrent event network: Autoregressive structure inferenceover temporal knowledge graphs. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6669–6683, 2020.

Yujia Li, Shiliang Sun, and Jing Zhao. Tirgn: Time-guided recurrent graph network with local-global historical patterns for temporal knowledge graph reasoning. In *Proceedings of the Thirty-First International Joint Conference on Artificial Intelligence (IJCAI)*, pages 2152–2158, 2022.

David Liben-Nowell and Jon Kleinberg. The link-prediction problem for social networks. *Journal of the American Society for Information Science and Technology*, 58(7):1019–1031, 2007.

Kunze Wang, Soyeon Caren Han, and Josiah Poon. Re-temp: Relation-aware temporal representation learning for temporal knowledge graph completion. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 258–269, 2023.

Jiapeng Wu, Meng Cao, Jackie Chi Kit Cheung, and William L. Hamilton. TeMP: Temporal message passing for temporal knowledge graph completion. In Bonnie Webber, Trevor Cohn, Yulan He, and Yang Liu, editors, *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 5730–5746, Online, November 2020. Association for Computational Linguistics. doi: 10.18653/v1/2020.emnlp-main.462. URL `https://aclanthology.org/2020.emnlp-main.462/`.

Cunchao Zhu, Muhao Chen, Changjun Fan, Guangquan Cheng, and Yan Zhan. Learning from history: Modeling temporal knowledge graphs with sequential copy-generation networks. *Proceedings of the AAAI Conference on Artificial Intelligence*, 2021.